Package caching is an optimization problem we should work on

Samuel Grayson

Suppose there is an arbitrary sequence of packages that you want to compile, possibly containing repeats. Compiling from source can be expensive, so your package manager can cache certain packages that may be reused in future request. In a package request,

- If the package is in the cache (called a "hit"), the compiled package is copied from the cache, incurring negligible cost.
- Otherwise (called a "miss"), the package manager has to request the dependencies of the package (recursively), and compile the package. Compiling incurs a cost associated with that package. The newly compiled package can be stored in the cache for future reuse. However, if the sum of the sizes of the packages in the cache exceeds the cache's total allowed size, the "cache eviction policy" must determine which packages to evict, until the size is not exceeded.

What's the optimal cache eviction policy that minimizes the total cost of compiling each request?

Note that the cost of compiling a package depends on whether or not its dependencies are in the cache (how far the recursion has to go to build the dependencies).

The nearest studied question in prior work assumes each package would have a fixed cost (see Prior Work for details). I think that **baseline would be easy to beat** by a policy that pays *some* attention to the dependency structure. This would lead to improvements for real-world programs (see Applications and Empirical Evaluation). This is not a case where we trade off one cost for another; improving the caching policy increases the cache hit ratio **for free.**

Prior work

Prior work on caching problems

In a general "caching problem":

1. We have "objects" each with a "size in cache" and a "cost to recompute".
2. We have a cache with a fixed capacity: the sum of the size of the objects in the cache must be less than that capacity.
3. We have a sequence of requests for a particular objects: if the requested object is in the cache, the cost is low; otherwise, the cost is higher.
4. The problem is to serve a sequence of requests to various objects with minimal cost.

Other "caching problems" make stronger assumptions, such as: all objects cost the same or all objects have the same size, such as page caching (i.e., for an OS kernel to manage pages of memory). Package caching relates to other caching problems thusly:

Caching problem	Cost to re-compute	Size in cache
Page caching	Fixed	Fixed
Variable-page caching	Fixed	Fn of cached object
Document caching	Fn of cached object	Fn of cached object
Package caching (proposed problem)	Fn of cache	Fn of cached object

Unlike prior caching problems, the cost to re-compute a package is not only dependent on the package itself, but also on which other packages are cached. E.g., if Ninja is cached, recompiling Chromium will be easier than if Ninja is not cached (would have compile Ninja in order to compile Chromium).

Prior work discusses differing assumptions about the known request trace:

- **Offline** assumes we know the entire trace of requests beforehand.
- **Online** assumes we only know the current request.
- **Steady-state** is a middle-ground, where we don't know the exact requests, but we know their distribution.

Importantly, the cache should be constant in the steady-state case, whereas it could change with every request in the online or offline case. Since the value of each package depends on the state of the cache, it is much easier to reason about the steady-state variant. However, prior work on page, document, or result caching usually focuses on offline or online, which makes more sense for those problems.

For **offline page caching**, the optimal policy is to evict the page whose next request is farthest in the future (Belady 1966).

An **online page caching** policy is said to be m competitive if there exists a b such that for all traces, the online policy incurs a total cost $mx + b$ for some

b, where x is the total cost of the optimal offline policy. Sleator and Tarjan show optimality of online LRU by showing that every online policy is at best s competitive, and LRU is exactly s competitive (Sleator and Tarjan 1985), where s is the size of the cache in pages.

Steady-state page caching is a trivial problem; all pages have the same cost and size but different frequencies. Fill the cache with pages with the largest frequencies.

Variable-page caching TODO

Offline document caching is NP-hard (TODO: cite), although there is a policy which can approximate the optimal to within a logarithmic factor (TODO).

For **online document caching**, in the special case of online document caching where the $c(p) \propto s(p)$, LRU is also optimal (Cao and Irani 1997). In the more general case, GD-SIZE (explained below) is optimal (*ibid.*), and it is k competitive where $k := s_c / \min_{p \in P} s(p)$. Cao and Irani identify the policies that always store the current document (at least initially) and then evict using one of the following policies:

- evict least recently used (LRU)
- evict least frequently used (LFU)
- evict largest (SIZE)
- greedy dual size (GD-SIZE): each package in the cache has a value, $v(p)$. When the package is initially stored, the value is set to the $c(p)/s(p)$. When we need to evict, remove the package with the minimum value, $p_{\min} := \arg\min_{p \in H(t)} v(p)$, reduce the value of all kept packages by $v(p_{\min})$. There is an optimization that optimizes away $|H(t)|$ subtractions (*ibid*).

It is possible that SIZE does well with documents in WWW scenarios because the cost of fetching a document increases with its size. This is not true of package caching.

The most apparently successful according to Cao and Irani is GD-SIZE. GD-SIZE assesses a "value" for each document, $c(p)/s(p)$, storing the most valuable (i.e., either easy-to-store documents or hard-to-fetch documents).

Steady-state document caching is exactly the same as 0-1 knapsack. Consider that we don't actually know $f(p)$, although we can estimate it from the requests at or before t, $\hat{f}_t(p)$. In expectation, the least-recent package (as in LRU) will also be the lowest $f(p)$, but that estimator is not as reliable as maintaining a counter of package popularity. We could be "unlucky" and a popular package happens to be least recently used.

Of course, we could get so "unlucky" and some popular package happens to not be popular in the first t requests. Although H does not nominally change with t in steady-state, the estimate of $\hat{f}_t(p)$ given the first t requests would. There is some aspect of "risk-averseness"; we may think that $f(p)$ is small, but if $c(p)$ is catastrophically large that we may want to cache it anyway.

TODO: Table of solns for (caching problem, request style).

Other related problems

Knapsack problem

All of these caching problems are also related to the knapsack problem: given a knapsack with a fixed weight capacity W and a set of n items with a weight w_i and a value v_i, choose items to maximize the sum of value of taken items such that the sum of the weights of taken items is less than W. The most relevant variant is called **0-1**, where the knapsack either contains 0 or 1 of each item; other variants allow the knapsack to contain multiple (bounded or unbounded) of each item.

Knapsack is NP-complete, as boolean satisfiability is reducible to chromatic number which is reducible to exact cover which is reducible to knapsack (Karp 1972).

However, certain "workarounds" make it possible to compute practical solutions:

- a **pseudo-polynomial time algorithm** exists termed the "**dynamic programming solution**". "Pseudo-polynomial" means that the algorithm in a polynomial time with the *value of the knapsack capacity* rather than the *number of items*
  - Compute the optimal solution for each capacity from 1 to W. At each step, the solution can be computed from the solutions to the smaller steps in $\mathcal{O}(1)$ (dynamic programming), rendering the whole algorithm in $\mathcal{O}(nW)$ and $\mathcal{W}$ space.
  - For caching, the knapsack capacity will probably be quite large, rendering this method impractical. Quantization is little help here, because packages likely have disparate sizes.
- a **log-linear 1/2-of-optimal approximate algorithm** exists termed "**decreasing density greedy (DDG)**".
  - Select the greatest remaining value-density items until the capacity is full, aka, which may require sorting the entire list of items, so it runs in $\mathcal{O}(n \log n)$ and $\mathcal{O}(n)$ space. The error would be maximized if there were a 51% capacity item worth 2$, which DDG would pick over a two 50% items each worth 1.90$, and no other items. In the limit, this situation would give a factor of two from optimal. Calvin and Leung 2003 show that given uniform random values and weights, the average-case performance is TODO. TODO Balas and Zemel have a linear version of this? Sort out the "log n"s in Sanhi's, table, and elsewhere.
- **two fully polynomial-time approximation scheme exist** (Sanhi 1975 and Ibarra and Kim 1975), that is, for any $k > 1$, there exists an approximate algorithm that attains at least $1 - \frac{1}{k}$ of the optimal value and runs in a polynomial time of the input and of k:

- Sanhi's algorithm tries every possible subset of $k-1$ or fewer, of which there are fewer than $(k-1)n^{k-1}$, and then proceeding with DDG, resulting in $\mathcal{O}(kn^k + n\log n)$ time. To prove the error bound, we can claim to have "guessed" the greatest $k-1$ values of the true optimal, and then used DDG to get half-of-optimal on the remaining items. If all the items were almost equal, the value of our approximate solution would have $\frac{k}{\text{opt}} + \frac{\text{opt}-k}{2}$ TODO. This algorithm is exponential in k, so it may be impractical for greater precision than $k = 2$ DDG.
- Ibarra-Kim's algorithm divides all w_i and W by a scaling factor $Q = \frac{\max_i v_i}{kn}$, discarding the remainder ($W' = \lfloor \frac{W}{Q} \rfloor$ and $w_i' = \lfloor \frac{W}{Q} \rfloor$). Now the dynamic programming can be applied. We have, $W \leq \sum_{i=0}^{n} v_i$ (otherwise the solution would be "take all items"), and $\sum_{i=0}^{n} v_i \leq n \max_i v_i$, therefore $W' = \frac{W}{Q} = \frac{knW}{\max_i v_i} \leq kn^2$, and the scheme runs in $\mathcal{O}(kn^3)$. Since we truncated to multiples of Q, the error would be at most the divisor times the number of addends, $nQ = \frac{\max_i v_i}{k}$. TODO

Algorithm	Time	Space	Factor of optimal value(approx. soln.) = factor * value(opt. soln.)
Dynamic programming	nW	W	1
DDG	$n \log n$	n	$\frac{1}{2}$
Sanhi	kn^k	n	$(1 - \frac{1}{k})$
Ibarra-Kim	kn^3	n	$(1 - \frac{1}{k})$

There are two *approximation schemes* which yields an approximate algorithm whose output is within a desired factor, $1 + 1/k$ for $k \in \mathbb{Z}+$, of the optimal total cost:

- Sanhi 1975, which is $\mathcal{O}(\frac{1}{\epsilon}|P|^{\frac{1}{\epsilon}+1} \log |P|)$ in time (I may have transcribed the variables incorrectly into this context)
- Ibarra and Kim 1975 which is $\mathcal{O}(k|P|^3 \log(|P| \max_{p \in P} c(p) \max_{p \in P} s(p)))$ in time (according to Garey and Johnson 1979 pg. 136; the primary sources is too confusing for me)

DDG is as good as $k = 1$ with time complexity $\mathcal{O}(|P| \log |P|)$. Since DDG is so much faster, it may be possible that higher k approximations are not worthwhile. But I am not sure.

DDG inspires an iterative solution to package caching:

1. Input a guess of the optimal cache function, called H.
2. Assess value $c'(p, H)/s(p)$ assuming the cache is H.
3. Create a new guess, H' which contains only the most value-dense packages until the cache is full.
4. Repeat back to 1 with $H := H'$ until H is "good enough".

However this strategy invites the following questions:

- Does H' actually have a lower total cost than H?
- Is H_{opt} an attractive fixed point of this process?
- Are there no other attractive fixed points (a.k.a. local minima)?
- How long does it take to converge?
- The total cost of this approximation is never worse than twice the total cost of the optimal; but what is the expected value of that ratio? The same question/answer for the pure 0-1 Knapsack problem (Calvin and Leung 2003) may shed light on this.

Submodular function maximization

Knapsack is a special case of submodular function maximization. A submodular function, g, is a function $2^P \to \mathbb{R}$ where for every $X, Y \subseteq P$ where $X \subseteq Y$, and every $p \in P \setminus Y$, we have $g(X \cup \{p\}) - g(X) \geq g(Y \cup \{p\}) - g(Y)$ (Vondrák lecture 2010, pg. 1). In other words, the smaller set, X, benefits at least as much from an additional element, p, as the larger set, Y, does. This property formalizes the idea of diminishing returns. Since it shows up a lot, we will use the symbol $\Delta_H(p)$ for "the marginal gain caused by adding p to H", $\Delta_H p = f(H \cup \{p\}) - f(H)$.

Indeed, the total savings, $-c'(p, H)$, over same request trace (online or offline case) or request distribution (steady-state) is submodular in H.

> **Proof**: Let $X \subseteq Y \subseteq P$ and $p \in P$.
>
> - Consider marginal gain of adding p to the cache X. The larger cache, $X \cup \{p\}$ will speed up compilation of p and all packages that have path of uncached ($\notin X$) dependencies originating at p.
> - Now, consider the marginal gain of p on Y. Again, we would save p and packages that have a path of uncached ($\notin Y$) dependencies originating at p.
>
> However, $X \subseteq Y$, so there are at least as many packages with uncached dependency paths $\notin X$ as there are $\notin Y$, resulting in a bigger marginal savings when increasing X to $X \cup \{p\}$ than increasing an already large set, Y, to $Y \cup \{p\}$. $\square$

In particular, $-c'(f, H)$ is a *monotonic*, submodular function, since a bigger cache is always better.

The constraint, $\sum_{p \in H} s(p)$, is a "knapsack" constraint; it is monotonic, modular function, due to being a linear function.

Precise formulation

Note: for this document, capital letter variables denote sets or functions onto sets, unless otherwise stated. 2^P will refer to the powerset of P, since the usual

powerset symbol, $\mathcal{P}$, looks too much like P. I will use Iverson-brackets, so $[x \in X]$ will evaluate to 0 or 1.

To make the most sense of prior work, we describe three variants of the package caching: offline, online, and steady-state.

The **(online or offline) package caching problem** is:

- **Given**:
  - A finite set of packages, P
  - Cost-of-building function, $c : P \to \mathbb{R}^+$. The function may measure the time, electricity, and/or compute resources of building from source, so long as the result is quantified as a real number; probably just walltime is a good start.
  - Package size function, $s : P \to \mathbb{R}^+$
  - Package dependency function, $D : P \to 2^P$. The digraph of packages induced by using D as a neighbor function is guaranteed to be acyclic.
  - Natural number, $n \in \mathbb{N}$, representing the number of time-points.
  - Sequence of requests, $r : P^n$, one for each time-point.
  - Total cache capacity, $s_c \in \mathbb{R}^+$
- **Let**:
  - $c'(p, H) : P \times 2^P \to \mathbb{R}$ be the cost of building-or-retrieving a package and any necessary dependents given cache-state, H. If the package is in the cache, the cost is zero. Otherwise, the cost is the cost of building the package plus sum of the costs of building-or-retrieving its dependents. In mathematical terms

$$c'(p, H) = [p \notin H] \left(c(p) + \sum_{p' \in D(p)} c'(p', H) \right).$$

  - $C'(r, H) : P^n \times \left(2^P\right)^n \to \mathbb{R}$ be the total cost of building-or-retrieving all requests, r_i, given the cache state at that time, H_i . In mathematical terms,

$$C'(r, H) = \sum_{i=0}^{n} c'(r_i, H_i).$$

- **Minimize** $C'(r, H)$
- **Over** cache contents, $H \to \left(2^P\right)^n$.
  - In the **offline version** of this problem, the policy can use $r_0, r_1, \ldots, r_i$ to compute H_i.
  - In the **online version**, it may only use the current request r_i and arbitrary state that is stored between requests.
- **Subject to**:

- Total size constraint, $\sum_{p \in H(t)} s(p) \leq s_c$
- Initially empty cache, $H_0 = \emptyset$
- The cache can only add the most recently compiled package, $H_i \subseteq H_{i-1} \cup \{p_{t-1}\}$

In the **offline** case, we know the entire trace ahead of time; in the **online** case, we know nothing about the trace ahead of time. We can also discuss a **steady-state** case, which is between the two extremes; we don't know the exact trace, but we know its distribution. We can make the following assumptions:

- **Independence of requests**. We are considering "top-level" requests (r_i in the previous problem), not requests to dependencies, so package dependency relationships don't threaten independence of top-level requests.
- **Identical distribution**. Even if the request distribution is time-dependent in the long-run (this year, Python 3.13 will be popular; last year it was 3.12), in the short-run, perhaps one week, we can assume requests are identically distributed.

These assumptions imply the r_i must be categorically distributed; each package has a frequency $f(p)$ which sums to one.

The **steady-state package caching problem** is:

- **Given** the quantities as in the exact package caching problem, *minus* the request trace or number of time-points, *plus* the frequency function, $f : P \to \mathbb{R}^+$ where $\sum_{p \in P} f(p) = 1$.
- **Minimize** *expected value* of total cost, $\mathbb{E}C'(f, H) = \sum_{p \in P} f(p) c'(p, H)$ where $c'(p, H)$ is as defined above.
- **Over** cache contents, $H \subset P$. Note we replaced time-dependent H_i with time-invariant H. If the distribution of requests is identical, then the re-use value of each package will be time-invariant, and therefore the cache should be time-invariant.

Perhaps we will treat $f(p)$ Pareto distributed; popularity contests often result in Pareto distributions because popularity draws more popularity.

Observations

- **Relation to vertex-cut**: $c'(p, H)$ sums up $c(p')$ for every node, p', between the requested package, p, and the immediately dominating vertex-cut contained in H.
- **Can compute** $c'(p, H) \in \mathcal{O}(\sum_{p \in P} |D(p)|)$ by the following algorithm:
  1. Topologically sort the DAG of P given by D.
  2. For each p in topological order, compute and store $c'(p, H)$, which involves taking a sum over $D(p)$. In this order, all recurrences will be

already computed when needed.

- **Can quickly compute cost over variations of** H by the following algorithm,
  1. Input H, $c'(p, H)$, and a variation H' whose cost $c'(p, H')$ we wish to comppute.
  2. Use the topological sort of $D(p)$ already computed (amortized $\mathcal{O}(1)$).
  3. For each package p in $H' \setminus H$, H' will save the cost of packages from p down to packages that were in the old H.
- **NP-hardness**
  1. A special case of steady-state package caching, where no packages depend on anything else, is reducible to 0-1 Knapsack. The value of each package in the knapsack problem would be set to the expected value of cost saved if that package were cached, $c'(p, H) = c(p)f(p)$. Therefore steady-state package caching is NP-complete. The best we could hope for is an approximate algorithm or approximation scheme.

Steady-state package caching is similar to 0-1 knapsack, except in steady-state package caching the expected value of value $(c'(p, H)f(p))$ depends on what packages are in the cache, (see definition of $c'(p, H)$). If the dependencies of p are not in the cache, the value of caching p is greater compared to the case where they are in the cache.

Applications

Functional package managers

Developing a more efficient caching policy could save CPU cycles for a lot users of functional package managers (e.g., Gamblin et al. 2015 (Spack), Dolstra et al. 2004 (Nix), Courtès 2013 (Guix)). Functional package managers have two relevant properties:

- Build from source; users can select any version, fork, and easily apply custom patches. This customizability is often necessary in high-performance computing environments: see Spack (Gamblin et al. 2015), Guix for HPC (Courtès and Wurmus 2015), and Nix for HPC (Bzeznik et al. 2017).
- Explicitly declaring all (build-time and run-time) inputs gives a high guarantee that the build process can be offloaded to any host meeting the build-time dependencies and the built artifact can be run on any host satisfying the run-time dependencies. Nix and Guix package managers enforce explicitly dependency declaration by build and run-tests in a sandbox containing *only* declared dependents (Nix 2.25 documentation, Guix documentation). While Spack does not have an explicit build sandbox on every build, it tests each package recipe in CI in a "blank" VM (Spack documentation).

The first property implies the utility of a cache (building from source is expensive), while the second implies its safety (cached builds will work on local host). All three of these package managers, Spack, Nix, and Guix have public and self-hostable build caches. However, space constraints will limit the effectiveness of the build cache. If the build cache is too ineffective, users will switch away to traditional package managers, resulting in less reproducibility.

I am a contributor to Spack and Nix, and I am in user forums for Guix. I think all three communities would welcome a minor change that improves the caching hit rate. The build cache themselves could collect and store package popularity and cost-to-compile. To prevent denial-of-service attacks, public build caches may distinguish trusted/voting and untrusted/non-voting users with authentication. Private build caches already have facilities to support authentication. If authentication is not feasible for public build caches, the popularity and cost-to-compile could be collected by other sources; popularity by running a separate popularity contest among trusted users, cost-to-compile likewise or in CI.

Today, Spack, Nix, and Guix build caches use "least recently used" cache eviction (TODO), which is not even optimal for document caching, since LRU is not cost-aware, so it would probably be easy to significantly improve.

Workflows

A workflow language (e.g., Make, Mölder et al. 2021 (Snakemake), Di Tomasso et al. 2017 (Nextflow), Jeremy Goecks et al. 2010 (Galaxy)) is a programming language where programs are structured as coarse-grained dataflow graphs. The graphs are usually acyclic, the nodes are usually scripts or containers, and the edges are usually files or parameters. Specialized workflows are commonly used in computational science experiments.

Workflows have the same two key properties of hermetic package managers:

- The nodes may be computationally intensive, mirroring the build-from-source nature of package managers.
- In order to make the workflow portable, some users try to encode every necessary input in the input-set (predecessors in the dataflow graph). Indeed, there is an option to run the nodes in a container with *only* those explicit inputs.

Therefore, reuse between workflows significantly save costs, especially on public clouds like Galaxy. In a Galaxy cloud, multiple users can log in and run each others' workflows through the web interface. While some users have unique workflows, due to the collaborative nature of science, many users want to reproducing those of others. Even unique workflows may reuse common components and data.

Hermetic build systems

Hermetic build systems (Buck, Buck2, Bazel, Pantsbuild) share the same two key properties as functional package managers. However, the motivation applies less strongly to hermetic build systems as functional package managers. Unrelated projects may choose slightly different versions of dependencies, limiting chances for reuse. When the source code is stored in a monorepo, caching the main branch's version of every package may be small enough for the cache and may provide good cache hit rate, since development branches start from the monorepo main branch. Furthermore, the problem is difficult to study empirically unless companies are willing to share their proprietary monorepos including commit history.

However, if the cache is too small to cache the whole monorepo build, a more complex policy may become relevant.

Empirical evaluation

While theoretical results are interesting, we would also test the most plausible cache policies on real-world data. We could mine Spack, Nix, and/or Guix package recipes from GitHub, which gives us not just one package dependency specification, but a revision history of specifications. We could interpret each commit as a request to build the packages of that recipe, weighted by the number of stars (proxy for the number of users who would want to build that recipe). For Nix, we may consider building the "default package", "all defined packages", and/or "default devshell".

For each unique package in the set (including different versions of the same package), we would need to capture the dependency graph and measure the time taken to compile the package and its dependencies.

From this dataset, we could simulate many package caching policies instantaneously.

Extensions

Do the results change when we consider multiple levels of caching? Naïvely, we might have each cache unaware that its consumer is another cache. It would infer the value of holding various items based on the frequency it observes at its own level.

- Is there a smarter way of doing that? Perhaps the lower level caches should only cache applications, and the higher level caches should only cache core libraries?
- Eventually caches would trade-off between consulting the higher-level cache versus building the necessary package locally. Where's the balance point?
- How would such a system work?

- Would we need a global list of each packages cost, or an index of what cache is likely to have what packages?
- Could caches join/leave the network dynamically?

Functional package managers already have multiple layers of cache by default: local machine cache and public cache. Institutions and organizations may introduce a third cache between the two.

Distributed hash table. TODO

Plan

1. I would round out the investigation of prior work, including variable size page caching, and steady-state cases.
2. Then, I would try adapting the simplest policies from prior work, especially DDG (which very similar to GD-SIZE) with iteration on H. I would only continue if I could show significant theoretical improvement over naïve GD-SIZE.
3. I would analyze any other generalizations of prior policies or new policies I can think of.
4. Should we consider running an integer-linear program solver using the same amount of resources as the algorithm used? Should we consider putting the output of the algorithm to an ILP set to run for 20% of the algorithm time?
5. I would try collecting data and doing empirical evaluations of the policies on package manager environment specifications found on public GitHub.